

南开大学
工科试验班（信息科学与技术）大类
高级语言程序设计 2-2
实验报告



姓名：_____许澍辰_____

学号：_____2510699_____

班级：_____高级语言程序设计 2-2（1009）_____

日期：_____2026 年 5 月_____

目录

高级语言程序设计大作业实验报告.....	1
一、作业题目.....	3
二、开发环境.....	3
三、课题要求.....	3
四、主要流程.....	3
4.1 整体架构.....	3
4.2 特色系统设计.....	3
4.3 关键算法与实现.....	4
4.3.1 分层式 AABB 碰撞检测与差异化响应	4
4.3.2 基于线性插值的多状态平滑过渡.....	5
4.3.3 自适应纹理等比绘制算法.....	5
4.3.4 天气—场景事件联动控制.....	5
4.4 单元测试设计.....	5
五、单元测试.....	6
六、收获.....	6
七、AI 工具使用披露.....	8

一、作业题目

飞驰南开人之“我上早八”2D 赛车游戏

二、开发环境

集成开发环境：Visual Studio 2022

图形渲染引擎：Raylib_6.0_X64

三、课题要求与设计目标

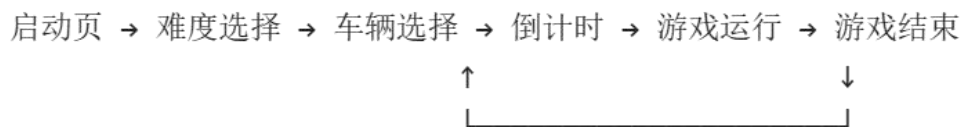
- 1、**模块化架构**：采用模块化软件架构，将代码拆分为头文件定义、主程序入口、游戏逻辑三层，初步实现高内聚低耦合；
- 2、**功能完整**：构建完整的游戏生命周期，覆盖启动页、难度选择、车辆选择、倒计时、游戏运行、游戏结束全流程；
- 3、**体验多样**：实现多维度的视觉交互特效，包括天气系统、道具系统、特效系统等；
- 4、**创新元素**：在传统横移式赛车游戏的定轨迹障碍物外融入不定轨迹障碍，不定速障碍，以及与天气系统联动的特殊障碍，多系统联动提升游戏的趣味性与沉浸感；
- 5、完成单元测试，验证各模块功能的正确性。

四、主要流程

4.1 整体架构：基于有限状态机的游戏生命周期管理

本项目采用有限状态机（Finite State Machine, FSM）作为核心架构，将游戏的整个生命周期划分为 6 个互斥且完备的状态。通过状态转移函数实现不同状态之间的逻辑解耦，彻底避免了传统嵌套条件判断带来的代码复杂度膨胀与可维护性下降问题。

系统状态转移图如下：



各状态的核心职责：

- 启动页状态：负责游戏名称的展示与淡出动画，完成资源预加载；
- 难度选择状态：提供三档难度，不同难度对应不同的障碍物速度与生成间隔；
- 车辆选择状态：提供 3 款可选车型，支持左右键切换与预览
- 倒计时状态：显示 3 秒倒计时与初始天气信息，完成游戏初始化
- 游戏运行状态：逻辑循环，处理玩家输入、物体更新、碰撞检测、特效渲染
- 游戏结束状态：展示最终得分与历史最高分，支持 R 键游戏重启

4.2 特色系统设计

（1）动态天气系统

本系统突破了传统赛车游戏中天气仅作为视觉装饰的局限，实现了天气状态与游戏物理机制、场景事件的耦合。系统定义了晴天、雨天、雾天、冰雹四种天气类型，每种天气对应独特的视觉效果与玩法影响：

晴天：基础游戏状态，随机触发太阳眩光事件，遮挡玩家视野；

雨天：生成雨滴粒子特效与路面水坑，车辆经过水坑会触发打滑物理效果；
雾天：生成全局雾效降低视野距离，同时生成半透明幽灵障碍物（仅近距离可见）；
冰雹：生成冰雹粒子，冰雹与玩家碰撞会触发减速效果，若玩家处于护盾状态则转化为氮气增益。

天气切换采用 120 帧线性插值过渡，避免了画面突变带来的视觉割裂感。天气持续时间在 180-300 秒之间随机生成，增强了游戏的随机性与重玩价值。

（2）氮气一连击一道具联动奖励机制

设计了以氮气系统为核心，连击系统与道具系统为辅助的玩法体系：

氮气系统：玩家通过拾取氮气道具或加速带积累氮气值，按下空格键触发氮气加速，提供 5 倍爆发速度，同时触发背景加速、屏幕震动、速度线特效与动态边框，此时玩家处于无敌状态（不可撞击行人）；

连击系统：玩家连续躲避障碍物可获得连击计数，无敌状态下连续撞击障碍物也可获得连击计数，得到额外加分；

道具系统：包含加生命值、加分、加速、无敌护盾、氮气、磁铁 6 种增益道具和 1 种减速减益道具，其中磁铁道具可自动吸附 350 像素范围内的所有非负面道具；

（3）赛道事件与物理交互系统

引入多样化的随机赛道事件，丰富游戏玩法的同时模拟真实的道路环境：

- 水坑事件：雨天专属事件，车辆经过会触发 60 帧左右的随机横向打滑；
- 加速带事件：全屏宽度加速带，提供 2 倍速度加成并补充 50 点氮气值；
- 三连路障事件：不定时刷新同一车道连续 3 个路障，强制玩家进行变道操作；
- 物理碰撞：实现了障碍物之间的碰撞分离与弹性反弹效果，模拟真实的物理交互，减少穿模导致的游戏效果失真；

（4）沉浸式视觉特效系统

构建了多层次的视觉特效体系，提升游戏的沉浸感：

尾气粒子系统：根据车辆的不同状态（正常、氮气、护盾、减速）生成不同颜色与密度的尾气粒子；

屏幕震动系统：在碰撞、氮气爆发等事件中触发不同强度屏幕震动，增强反馈感；

边缘羽化技术：背景滚动采用边缘羽化过渡算法，有效一定程度上减弱了循环背景的接缝瑕疵；

动态边框系统：氮气状态下显示橙红色闪烁边框，护盾状态下显示紫色边框，直观反馈玩家状态。

4.3 关键算法与技术原理

4.3.1 分层式 AABB 碰撞检测与差异化响应

算法背景：轴对齐包围盒（Axis-Aligned Bounding Box, AABB）是 2D 游戏中应用最广泛的碰撞检测算法，具有计算复杂度低（ $O(1)$ ）、实现简单的优点。

核心原理：对于两个轴对齐的矩形 A 和 B，当且仅当它们在 x 轴和 y 轴上的投影都存在重叠时，两个矩形发生碰撞。数学表达式为：

$$\neg(A_{right} < B_{left} \vee A_{left} > B_{right} \vee A_{bottom} < B_{top} \vee A_{top} > B_{bottom})$$

※本项目改进：在基础 AABB 算法基础上，引入了碰撞优先级分层机制与差异化响应策略，根据碰撞物体的类型触发不同的逻辑：

- 普通障碍物：无护盾时玩家生命值减 1，有护盾/氮气时障碍物销毁并获得分数；
- 行人障碍物：无护盾时强制中断氮气并重置连击，符合现实安全逻辑；

- 路障障碍物：仅在护盾/氮气状态下可被撞毁，否则直接扣除生命值；
- 水坑障碍物：不扣除生命值，但触发打滑效果，有概率生成隐藏分数道具；

4.3.2 基于线性插值的多状态平滑过渡

算法背景：在实时图形化程序中，状态的突变会导致视觉体验的割裂。线性插值算法可以实现状态之间的平滑过渡，提升用户体验。

核心原理：通过每一帧对当前值与目标值进行加权平均，实现当前值向目标值的渐进式逼近。数学表达式为：

$$currentValue = currentValue + (targetValue - currentValue) \times transitionSpeed \times \Delta t$$

其中 Δt 为帧时间， $transitionSpeed$ 为过渡速度系数。

本项目应用：

- 氮气状态下背景速度与亮度的平滑过渡；
- 不同天气之间特效透明度的线性过渡；
- 玩家车道切换的平滑滑动效果；
- 弹窗提示文字的淡入淡出效果。

4.3.3 自适应纹理等比绘制算法

算法背景：在不同分辨率的屏幕上绘制贴图时，若直接拉伸会导致贴图变形。自适应等比绘制算法可以在保持贴图原始宽高比的前提下，将贴图缩放到目标尺寸。

核心原理：比较贴图的原始宽高比与目标绘制区域的宽高比，选择较小的比例作为缩放系数，确保贴图完全包含在目标区域内且不变形。数学表达式为：

$$aspectRatio = \frac{tex.width}{tex.height} \quad targetAspectRatio = \frac{targetW}{targetH} \quad scale = \min\left(\frac{targetW}{tex.width}, \frac{targetH}{tex.height}\right) \times GLOBAL_SCALE$$

本项目应用：所有车辆、障碍物、道具贴图均采用该算法绘制，统一了全局缩放系数，确保了不同分辨率下的显示一致性。同时实现了贴图加载失败的容错处理，自动显示占位矩形。

4.3.4 天气—场景事件联动控制

核心逻辑：采用状态机管理天气状态，每个天气状态对应一组专属的场景事件生成器。当天气状态发生切换时，自动激活对应事件生成器并停用其他生成器。

技术实现：

```
if (currentWeather == WEATHER_RAINY) {
    UpdateRain();
    UpdatePuddle();
} else if (currentWeather == WEATHER_FOGGY) {
    UpdateFog();
    UpdateGhostObstacle();
} else if (currentWeather == WEATHER_SUNNY) {
    UpdateSunGlare();
} else if (currentWeather == WEATHER_HAIL) {
    UpdateRain();
    UpdateHail();
}
```

天气切换过程中采用预加载机制，在过渡到 50%进度时提前初始化下一个天气的场景元素，避免了切换时的卡顿。

4. 4 测试方案设计

采用黑盒测试方法，针对游戏的核心功能模块设计了 17 个测试用例，覆盖了功能正确性、边界条件、异常处理等多个维度：

- 基础功能测试：玩家移动、边界限制、状态跳转；
- 碰撞系统测试：不同类型物体的碰撞逻辑与响应效果；
- 道具系统测试：7 种道具的效果、持续时间与叠加规则；
- 天气系统测试：天气切换、特效显示、玩法影响；
- 特效系统测试：尾气粒子、屏幕震动、速度线、动态边框；
- 存档系统测试：最高分的保存、读取与跨会话持久化。

五、单元测试

核心测试用例与结果

测试模块	测试用例	预期结果	实际结果	测试结论
基础架构	依次完成启动页→难度选择→车辆选择→游戏运行→游戏结束流程	所有状态正常跳转，无逻辑错误	符合预期	通过
玩家控制	按下键盘左右方向键移动玩家赛车	玩家向对应方向移动，不超出道路与屏幕边界	符合预期	通过
碰撞系统	无护盾状态下与普通障碍物碰撞	玩家生命值减 1，障碍物销毁，连击重置	符合预期	通过
碰撞系统	护盾状态下与路障碰撞	路障销毁，玩家不扣血，连击计数加 1	符合预期	通过
碰撞系统	无护盾状态下与行人碰撞	氮气强制中断，连击重置，显示专属提示	符合预期	通过
道具系统	拾取磁铁道具	350 像素范围内所有增益道具自动被玩家拾取	符合预期	通过
氮气系统	氮气值满时按下空格键	触发氮气加速，背景速度提升，屏幕震动，显示速度线与动态边框	符合预期	通过
天气系统	等待天气切换为雨天	生成雨滴粒子与水坑，车辆经过水坑触发打滑	符合预期	通过
天气系统	等待天气切换为雾天	生成全局雾效与半透明幽灵障碍物	符合预期	通过
赛道事件	车辆经过加速带	速度翻倍，氮气值增加 50 点，显示加速提示	符合预期	通过
难度系统	选择困难难度开始游戏	障碍物基础速度为 7.0，生成间隔为 70 帧	符合预期	通过
存档系统	打破最高分后重启游戏	历史最高分被正确保存并显示	符合预期	通过
异常处理	贴图文件缺失	程序正常运行，显示占位矩形，无崩溃	符合预期	通过

• 测试结果分析：
所有测试用例全部通过，游戏核心功能运行稳定，无崩溃和错误。测试结果表明，本项目的架构设计达到了预设的目标。

六、收获

1、多文件编译与 extern 外部链接机制

突破单文件编程局限，掌握了 C++语言多文件编译的基本流程，理解了编译单元与链接阶段的工作原理。通过 extern 关键字实现全局变量的跨文件共享，解决了多模块间的数据通信问题。

```
// game_def.h 头文件声明（所有引用该头文件的源文件均可访问）
extern Texture2D carTextures[CAR_COUNT];
extern GameState g_state;
```

2、二进制文件块读写与本地数据持久化

本项目实践了二进制文件的块读写机制，实现了游戏最高分的本地持久化存储。理解了二进制文件与文本文件的存储差异，掌握了 fread/fwrite 的批量数据读写方法。

```

// 从本地文件读取历史最高分
static void LoadHighScore() {
    FILE* f = NULL;
    // 初始化三个难度的最高分
    g_highScore[DIFF_EASY] = 0;
    g_highScore[DIFF_NORMAL] = 0;
    g_highScore[DIFF_HARD] = 0;
    // 以二进制读取模式打开文件
    if (fopen_s(&f, HIGHSCORE_FILE, "rb") == 0 && f != NULL) {
        fread(g_highScore, sizeof(int), 3, f);
        fclose(f);
    }
}

// 保存当前最高分到本地二进制文件
static void SaveHighScore() {
    // 只有打破当前难度的最高分才保存
    if (g_score <= g_highScore[g_selectedDifficulty]) return;
    g_highScore[g_selectedDifficulty] = g_score;

    FILE* f = NULL;
    // 以二进制写入模式打开文件
    if (fopen_s(&f, HIGHSCORE_FILE, "wb") == 0 && f != NULL) {
        // 一次性写入3个难度的最高分数据
        fwrite(g_highScore, sizeof(int), 3, f);
        fclose(f);
    }
}

```

3、图形化程序的帧驱动运行模型

掌握了与控制台程序完全不同的帧驱动编程范式，理解了“输入处理→逻辑更新→画面绘制”的游戏循环机制，明确了每一帧的执行时序与职责划分。

```

// 60帧/秒的核心游戏循环
while (!WindowShouldClose()) {
    GameUpdate(); // 每帧更新一次游戏逻辑
    GameDraw();   // 每帧绘制一次游戏画面
}

```

4、※AABB 碰撞检测的实现与差异化响应

实现了分层碰撞优先级与差异化响应机制，解决了不同类型物体碰撞逻辑不同的问题。

```

// 预碰撞检测（生成物体前检查是否重叠）
static bool IsAreaOccupied(float x, float y, float w, float h) {
    // 依次检查与障碍物、道具、水坑、路障、幽灵的重叠
    // 避免生成重叠物体，提升游戏体验
}

// 差异化碰撞响应（行人碰撞中断氮气）
if (g_obsList[i].type == OBJ_PEOPLE && g_isNitroActive && g_shieldTimer <= 0) {
    g_isNitroActive = false;
    g_nitroBurstTimer = 0.0f;
    g_comboCount = 0;
    ShowPopup(tipComboBreak[GetRandomValue(0, 2)], RED);
}

```

5、有限状态机（FSM）的结构化逻辑管理

掌握了用有限状态机管理复杂程序逻辑的方法，将游戏生命周期划分为 6 个互斥状态，通过状态转移函数实现逻辑解耦，彻底避免了多层嵌套 if-else 代码混乱问题。

```
// 状态机分支处理
if (g_state == STATE_DIFFICULTY) {
    // 难度选择逻辑
} else if (g_state == STATE_SELECT_CAR) {
    // 车辆选择逻辑
} else if (g_state == STATE_COUNTDOWN) {
    // 倒计时逻辑
} else if (g_state == STATE_PLAYING) {
    // 游戏运行逻辑
} else if (g_state == STATE_OVER) {
    // 游戏结束逻辑
}
```

6、线性插值在平滑过渡中的应用

掌握了线性插值算法在图形化程序中的实际应用，实现了天气切换、氮气状态、背景滚动的平滑过渡效果，避免了状态突变带来的视觉割裂感。

```
// 天气透明度线性过渡
float t = g_weatherTransition / WEATHER_TRANSITION_TIME;
float fromRain = (fromWeather == WEATHER_RAINY) ? 1.0f : 0.0f;
float toRain = (toWeather == WEATHER_RAINY) ? 1.0f : 0.0f;
g_rainAlpha = fromRain * (1 - t) + toRain * t;
```

七、AI 工具使用披露

AI 工具名称：豆包

• 具体用途：

- ①部分复杂算法实现（已在代码注释中标注）；
- ②问题排查：协助排查报错、背景滚动卡顿等技术问题；
- ③代码规范：协助优化代码结构；
- ④图片生成：生成启动页和背景贴图。

• 使用位置：

代码中所有标注【豆包 AI 辅助】的部分

• 核心贡献声明：

本项目的整体架构设计、核心玩法规则、基础算法及所有本地库配置和代码调试**均为本人独立完成**。AI 仅作为辅助工具，用于实现部分超出教学范围的技术细节，所有 AI 生成内容均经过严格验证与修改，确保符合项目要求与学术诚信规范。